GCN for Molecule Dipole Moment Prediction

Ksenia Krasikova[1*†], **Payal Priyadarshini**[1*†], **Philipp Harlos**[1*†]

***For correspondence:**
ksek00@zedat.fu-berlin.de (KK);
payap02@zedat.fu-berlin.de (PP);
harlop97@zedat.fu-berlin.de (QQ)

[†]These authors contributed equally to this work

[1]Department of Mathematics and Informatics, FU Berlin, Berlin, DE

Abstract

Goal of the project: To implement a Graph Convolutional Network (GCN) to the problem of graph-level prediction of molecular properties using the QM9 dataset, and predicting the electric dipole moment.
Main results: A trained GCN model for predicting the electric dipole moment of molecules and evaluation of the model's performance with Mean Squared Error (MSE), Mean Abosulte Error(MAE) and Huber loss functions.
Personal key learnings: An introduction and implementation of GCN using JAX ecosystem tools like Jraph and Flax, by graph representation, model implementation, and evaluation techniques for molecular property prediction, and knowing the importance of permutation properties (equivariance and invariance) in graph neural networks.
Estimated working hours: 10
Project evaluation: 3
Number of Words: 1427

Goal

This week's project focuses on the implementation of a Graph Convolutional Network (GCN). Using a provided code template and the QM9 benchmarking dataset, the objective is to train a GCN model to predict the dipole moment of molecules. In this context, each molecule is represented as a graph, where atoms serve as nodes and chemical bonds as edges.

Data

For this project, we use the QM9 dataset, which does not require manual download as it is directly accessible via **torch_geometric.datasets.QM9**. This greatly simplifies preprocessing, since the data is already provided as molecular graphs in a suitable structure. The dataset consists of approximately 133,000 molecules, each represented as a graph with nodes and edges. Nodes correspond to atoms, described by 11 features including one-hot encoded atom type, atomic number and other atomic properties. Edges represent chemical bonds and are described by a one-hot encoded vector for bond type. In addition to node and edge features, each graph also contains the adjacency matrix, 3D coordinates for each atom, and 19 target properties describing various chemical properties of the molecule.

Pre-Processing

First, after loading the data, all features except for the dipole moment are dropped, as we only focus on this regression target for this project. Then, the data is split into training, test and validation data, each with 1000 molecules.
The data is initially loaded in the **torch_geometric.data.Data** format, which represents each molecule

as a graph with associated node and edge features. However, for subsequent processing and model training, the data must be converted into the **jraph.GraphsTuple** format. To achieve this, each attribute, such as node features, edge features, edge indices (adjacency), and graph-level targets, is manually transferred from the PyG object to the new format. This transformation process is integrated into a custom batching function for the **DataLoader**. During batching, multiple molecular graphs are combined into a single batch by merging their features and connectivity information. Additional padding is added both for nodes and for the number of graphs in the batch. As a result, each batch is one unified, padded graph structure.

Methods

Layers and Model

The core building block of the model is a custom graph convolutional layer (GCL), implemented according to the convolutional message-passing paradigm introduced in the lecture. Each GCL includes two learnable components: psi and phi, both being neural network layers. In the final model, both are implemented as single-layer neural networks, specifically dense layers with 128 output units. As a result, each GCL contains two such layers, each projecting into a 128-dimensional feature space. The psi function computes messages from all sender nodes x_j to their corresponding receiver nodes x_i. The phi function then updates each node's (x_i) representation by combining its original features (atom-level features) with the aggregated messages from its neighbors, transforming it into h_i. Message aggregation is performed using a sum operator. A mathematical equation representing this can be seen in Equation 1. The message-passing procedure allows the network to build a more expressive feature space over successive layers: node representations are no longer limited to intrinsic atom features but are gradually enriched with structural context from the surrounding graph.

$$x_i' = \phi\left(x_i, \sum_{j \in \mathcal{N}(i)} \psi\left(x_j\right)\right) \tag{1}$$

The GCN model uses the GCL layer function to construct the model, using custom number of GCL layers and size of the hidden feature vector (number of neurons in the dense layers). After passing through the convolutional layers, each node holds a richer representation. However, since the goal is to make a graph-level prediction, these node-level embeddings must be condensed into a single graph-level representation. To achieve this, we applied a mean pooling operation over all node features in a graph. This produces one fixed-size vector (size of the hidden feature vector) for every graph. Finally, this graph embedding is passed through a dense layer to make a scalar prediction, in our case, representing the electric dipole moment of the molecule.

Equivariance and Invariance

To verify the geometric correctness of our implementation, we tested the model for two key properties: **layer-level equivariance** and **model-level invariance**. Both were assessed using a permuted version of a graph. Permuting here means randomly reordering nodes while preserving edge connections, so the graph structure remains unchanged—only the internal representation differs. This is analogous to rotating a molecule without altering its structure.

A layer is permutation-equivariant if permuting the input nodes results in the output being permuted in the same way. We tested this for our custom graph convolutional layer (GCL) using a single graph from the training set. We defined psi and phi as dense layers with eight output units for quick testing. After permuting the nodes and adjusting edge indices accordingly, we passed both the original and permuted graphs through the same GCL. We then permuted the output of the original graph and compared it to the output of the permuted graph. The results matched up to a numerical tolerance of 1e-5, confirming the expected equivariant behavior.

A model is permutation-invariant if its output remains unchanged under node permutations. This

is essential for graph-level prediction, where output should not depend on node order. We tested this for the full GCN by using a small model with two GCLs and eight-dimensional hidden features. As in the equivariant test, we permuted the graph and updated its edge indices, then applied the model to both versions. The scalar outputs were identical, confirming the GCN's permutation invariance.

Training and Evaluation

We built our final model by stacking four graph convolutional layers (GCLs). The first layer transforms the initial 11-dimensional node (atom) features into 128-dimensional representations; the remaining three layers preserve this dimensionality. Model parameters were initialized using a random key and a sample batch from the training set. We trained the model for 20 epochs using the Adam optimizer with a learning rate of 1×10^{-4}, updating parameters based on the gradient of a chosen loss function. To compare performance, we trained the model using three loss functions. As a baseline, we used mean squared error (MSE), a standard choice for regression tasks like predicting molecular dipole moments. We also tested mean absolute error (MAE), which penalizes all errors equally and is less sensitive to outliers, and the Huber loss, which blends the behavior of MSE and MAE for improved robustness. After each epoch, we evaluated model performance on the validation set to monitor generalization. While we tracked validation loss, we did not use it to adjust training (e.g., early stopping), due to time constraints. Final performance was measured on a separate test set using all three loss functions. As expected, training loss generally decreased over time, and validation loss helped assess how well the model generalized to unseen data.

Results & Discussion

To create discussion, we compare different loss functions for training the GCN. We use MSE as the baseline of our analysis. As shown in ***Figure A1***, the training and validation losses both smoothly decreased and plateaued around epoch 7, after which no further significant improvement was observed. This suggests that the model had largely converged by that point, and training for additional epochs had limited effect. Interestingly, the validation loss initially dropped more steeply than the training loss, but then was slightly more increased compared to the training loss for the last 10-14 epochs.

In contrast, the mean absolute error (MAE), as shown in ***Figure A2***, fluctuated throughout all 20 epochs, however, showing a downward trend. This behavior suggests that MAE may not be well-suited for this particular training setup, or that a significantly longer training schedule might be required to stabilize it. However, since the fluctuations do not appear to diminish over time, it seems more likely that MAE is simply less stable with our model or data in this case, especially when compared to the smoother convergence observed with MSE. But the training process using the MAE loss yields a better test error than training with MSE loss: the test loss with MSE is at 4.63 where the test loss with MAE is at 2.35.

As for Huber loss in ***Figure A3***, both training and validation losses show an overall decrease over time, but similar fluctuations as in the MAE error behaviour. However, out of the three, the model trained with Huber loss shows the best performance on unseen data with a loss of approximately 1.23. Although both MAE and Huber loss showed greater fluctuations during the training process, both achieve better results on unseen data than the baseline MSE loss. Although the model trained with MSE loss converged smoothly and faster, it yielded worse results on the test data. This suggests that although showing better convergence, MSE is less suited for this prediction, as it penalizes large errors strongly, which can lead to overfitting due to outliers.

Contributions

1. Baseline code implementation, writing Goal, Data and Pre-proccessing section, pre-writing Methods sections by Ksenia Krasikova.

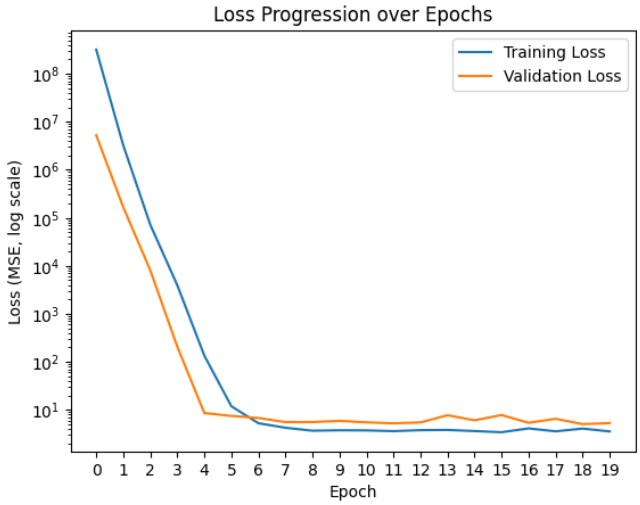
2. Worked on Huber Loss model implementation, partly writing Methods Section and Result/Discussion and minor changes by Payal Priyadarshini.
3. Worked on baseline and MAE model implementation, writing Method Section and partly Result/Discussion (MAE loss function) by Philipp Harlos.

References

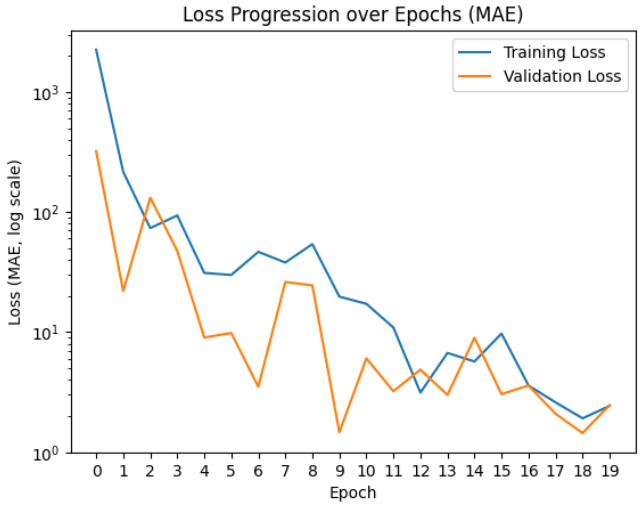
Acharya N, Choosing Between Mean Squared Error (MSE) and Mean Absolute Error (MAE) in Regression: A Deep Dive — nirajan.acharya777;. [Accessed 09-07-2025]. https://medium.com/@nirajan.acharya777/choosing-between-mean-squared-error-mse-and-mean-absolute-error-mae-in-regression-a-deep-dive-c16b4eeee603.

Chen B, Understanding Huber Loss function: Insights from Applications — devcharlie2698619;. [Accessed 09-07-2025]. https://medium.com/@devcharlie2698619/understanding-huber-loss-function-insights-from-applications-5c1c5145d2c4.

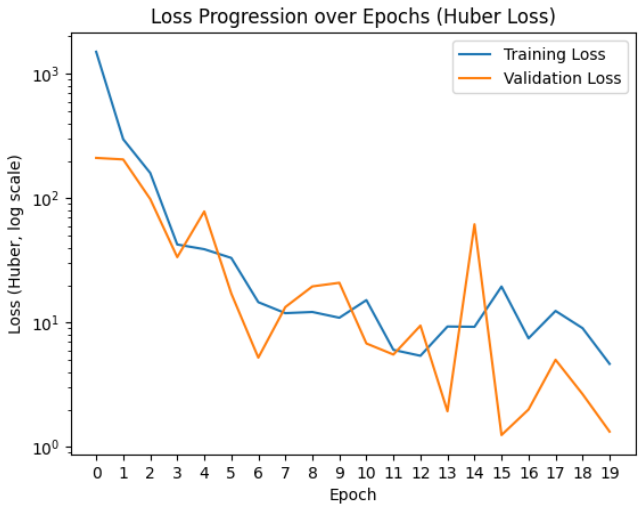
Supplementary Figures



Appendix 0—figure A1. Training and validation loss curves (MSE, log scale) for the GCN model over 20 epochs.



Appendix 0—figure A2. Training and validation loss curves (MAE, log scale) for the GCN model over 20 epochs.



Appendix 0—figure A3. Training and validation loss curves (Huber Loss, log scale) for the GCN model over 20 epochs.